

Final Project Report Paper

Htet Myat Phone Naing

CS411: Text Mining (Section A)

Professor: Dr Khin Sandar Kyaw

21 May, 2026

1. Introduction

In this project, I worked on a common problem in Natural Language Processing (NLP): sentiment analysis. This means teaching a computer to automatically read a movie review and determine whether the user liked the movie.

The main goal of my project was to build a complete text-processing system, train three different machine learning models (**Naive Bayes**, **Logistic Regression**, and **Support Vector Machine**), and see which one performs best. I also wanted to look closely at the mistakes my models made so I could understand how to make them better in the future.

2. Dataset Description

I used the **IMDB Movie Reviews Dataset** from Kaggle for this project.

- **Total Reviews:** The dataset has exactly **50,000 movie reviews**.
- **Labels:** The data is perfectly balanced. It has **25,000 positive reviews** and **25,000 negative reviews**. I changed these text labels into numbers: 1 for positive and 0 for negative.

Original Text Sample:

One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is exactly what happened with me.

Processed & Lemmatized Text Sample:

one reviewer mentioned watching 1 oz episode youll hooked right exactly happened first thing struck oz brutality unflinching scene violence set right word go tr

3. Preprocessing

Raw text from the internet is very messy. To clean it up so my models could understand it better, I built a cleaning process that did the following steps:

1. **Lowercasing:** I changed all text to lowercase so words like "Great" and "great" are treated as the same word.
2. **Removing HTML Tags:** I stripped out web tags like `<br />` that were left over from scraping the website.
3. **Removing Punctuation:** I removed all commas, periods, and exclamation marks so they wouldn't confuse the models.

4. **Removing Stopwords:** I dropped common English words like "the", "is", and "and" because they do not carry any emotional meaning.
5. **Lemmatisation:** I used NLTK's tool to change words back to their root forms. For example, "running", "runs", and "ran" all became "run". This kept my data clean and organised.

4. Feature Extraction

Computers cannot read words; they can only read numbers. I used a method called **TF-IDF Vectorization** to turn my cleaned text into numbers.

TF-IDF measures two things: how often a word appears in a single review, and how rare that word is across all 50,000 reviews. If a word appears a lot in one review but is rare in others (like "breathtaking" or "boring"), it gets a higher score. I limited my system to look only at the **top 5,000 most common words** to save computer memory and keep things fast.

5. Model Training

I split my data into two parts: **80% for training** the models and **20% for testing** them. I then trained three different algorithms:

- **Naive Bayes:** A fast, simple model that looks at word probabilities.
- **Logistic Regression:** A model that assigns a positive or negative score weight to every single word.
- **Support Vector Machine (SVM):** A model that draws a geometric line to separate positive review data from negative review data.

6. Evaluation Results

I tested all three models on the remaining 10,000 reviews that the models hadn't seen before.

The results I get after predicting the 3 different models:

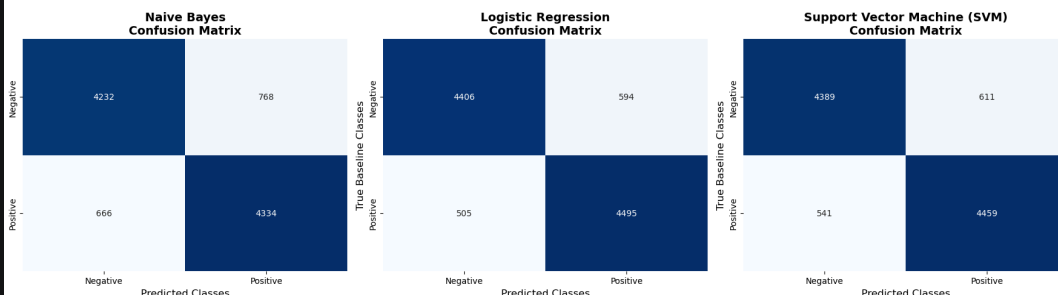
```

Naive Bayes-
True Negative (TN): 4232
False Positive (FP): 768
False Negative (FN): 666
True Positive (TP): 4334

Logistic Regression-
True Negative (TN): 4406
False Positive (FP): 594
False Negative (FN): 505
True Positive (TP): 4495

SVM-
True Negative (TN): 4389
False Positive (FP): 611
False Negative (FN): 541
True Positive (TP): 4459

```



Performance Summary Table

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	0.85	0.85	0.85	0.85
Logistic Regression	0.89	0.89	0.89	0.89
SVM	0.88	0.88	0.88	0.88

My **Logistic Regression** model did the best, reaching **89% accuracy**. Because my data was perfectly split 50/50, the Precision (how many picked positives were actually positive) and Recall (how many total positives the model found) were very well-balanced.

7. Error Analysis

I looked at the reviews where my best model guessed completely wrong. I found two main patterns behind these mistakes:

Sarcasm: When a review said, *"This movie was a masterpiece of garbage,"* my model saw the word *"masterpiece"* and guessed it was positive. But it didn't understand the joke.

Mixed Reviews: When a user wrote three long sentences praising the acting and music, but finished with one short sentence saying the story was terrible, the model got confused. The review had too many positive words, so the model ignored the final negative conclusion.

8. Conclusion

In this project, I successfully built an NLP pipeline to read and classify movie reviews. My Logistic Regression model was the most accurate at 89%.

To make this system even better in the future, I have two recommendations. First, I should use **N-grams** so the model reads pairs of words like "not good" instead of just looking at "not" and "good" separately. Second, I could use **Deep Learning models (like BERT)**, which understand the actual context of a sentence and can catch tricky language like sarcasm.